NAMA : NOLDY FERNANDES BANA (257111069)

KELAS/PRODI : C/ PENDIDIKAN INFORMATIKA

MATA KULIAH : STRUKTUR DATA

```
class Mahasiswa:
    def __init__(self, nim, nama, jurusan):
        self.nim = nim
        self.nama = nama
        self.jurusan = jurusan

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None # Hapus baris ini kalau mau jadi Single Linked List

class LinkedListMahasiswa:
    def __init__(self, double=True):
        self.head = None
        self.tail = None
        self.double = double # True = Double, False = Single

    # 1. TAMBAH DI AWAL
    def tambah_awal(self, nim, nama, jurusan):
        mhs = Mahasiswa(nim, nama, jurusan)
        node = Node(mhs)
        if not self.head:
            self.head = self.tail = node
        else:
            node.next = self.head
            if self.double:
                self.head.prev = node
            self.head = node
        print(f"-> {nama} ditambah di AWAL")

    # 2. TAMBAH DI AKHIR
    def tambah_akhir(self, nim, nama, jurusan):
        mhs = Mahasiswa(nim, nama, jurusan)
        node = Node(mhs)
        if not self.head:
            self.head = self.tail = node
        else:
            self.tail.next = node
            if self.double:
                node.prev = self.tail
            self.tail = node
```

```
        print(f"-> {nama} ditambah di AKHIR")

    # 3. TAMBAH DI TENGAH (URUT NIM)
    def tambah_urut_nim(self, nim, nama, jurusan):
        mhs = Mahasiswa(nim, nama, jurusan)
        node = Node(mhs)

        if not self.head or nim < self.head.data.nim:
            self.tambah_awal(nim, nama, jurusan)
            return

        curr = self.head
        while curr.next and curr.next.data.nim < nim:
            curr = curr.next

        # Cek duplikat NIM
        if curr.data.nim == nim:
            print(f"!! NIM {nim} sudah ada!")
            return

        node.next = curr.next
        curr.next = node
        if self.double:
            if node.next:
                node.next.prev = node
            node.prev = curr
            if curr == self.tail:
                self.tail = node
        print(f"-> {nama} ditambah urut NIM")

    # 4. HAPUS BERDASARKAN NIM
    def hapus(self, nim):
        if not self.head:
            print("List kosong!")
            return

        curr = self.head
        while curr and curr.data.nim != nim:
            curr = curr.next

        if not curr:
            print(f"NIM {nim} tidak ditemukan!")
            return

        if curr == self.head:
            self.head = curr.next
            if self.head and self.double:
                self.head.prev = None
```

```
            if curr == self.tail:
                self.tail = None
        elif curr == self.tail:
            self.tail = curr.prev if self.double else self._get_prev(curr)
            if self.tail:
                self.tail.next = None
        else:
            prev_node = curr.prev if self.double else self._get_prev(curr)
            prev_node.next = curr.next
            if self.double and curr.next:
                curr.next.prev = prev_node

        print(f"-> Data NIM {nim} berhasil dihapus")

    def _get_prev(self, node_target): # Helper khusus Single Linked List
        curr = self.head
        while curr and curr.next != node_target:
            curr = curr.next
        return curr

    # 5. CARI BERDASARKAN NIM (Linear Search)
    def cari(self, nim):
        curr = self.head
        posisi = 0
        while curr:
            if curr.data.nim == nim:
                print(f"DITEMUKAN di posisi {posisi}: {curr.data.nim} |
{curr.data.nama} | {curr.data.jurusan}")
                return curr.data
            curr = curr.next
            posisi += 1
        print(f"NIM {nim} tidak ditemukan")
        return None

    # 6. TAMPILKAN SEMUA
    def tampilkan(self):
        if not self.head:
            print("Belum ada data mahasiswa")
            return
        print("\n--- DAFTAR MAHASISWA ---")
        print(f"{'NIM':<10} {'Nama':<15} {'Jurusan'}")
        print("-" * 40)
        curr = self.head
        while curr:
            print(f"{curr.data.nim:<10} {curr.data.nama:<15}
{curr.data.jurusan}")
            curr = curr.next
```

```
        print("-" * 40 + f" [Mode: {'DOUBLE' if self.double else 'SINGLE'}
Linked List]\n")

# --- PROGRAM UTAMA (MENU) ---
if __name__ == "__main__":
    print("Pilih implementasi:")
    print("1. Single Linked List")
    print("2. Double Linked List")
    pilih = input("Pilih (1/2): ")
    is_double = True if pilih == "2" else False

    ll = LinkedListMahasiswa(double=is_double)

    # DATA AWAL OTOMATIS
    data_awal = [
        (257111056, "Ryan", "Informatika"),
        (257111064, "Moman", "Sistem Informasi"),
        (257111069, "Noldy", "Teknik Komputer"),
        (257111057, "Rafika", "Kebidanan")

    ]
    for nim, nama, jurusan in data_awal:
        ll.tambah_urut_nim(nim, nama, jurusan)

    print("\n[3 Data awal berhasil ditambahkan, silahkan pilih menu 6 untuk
tampilkan]")

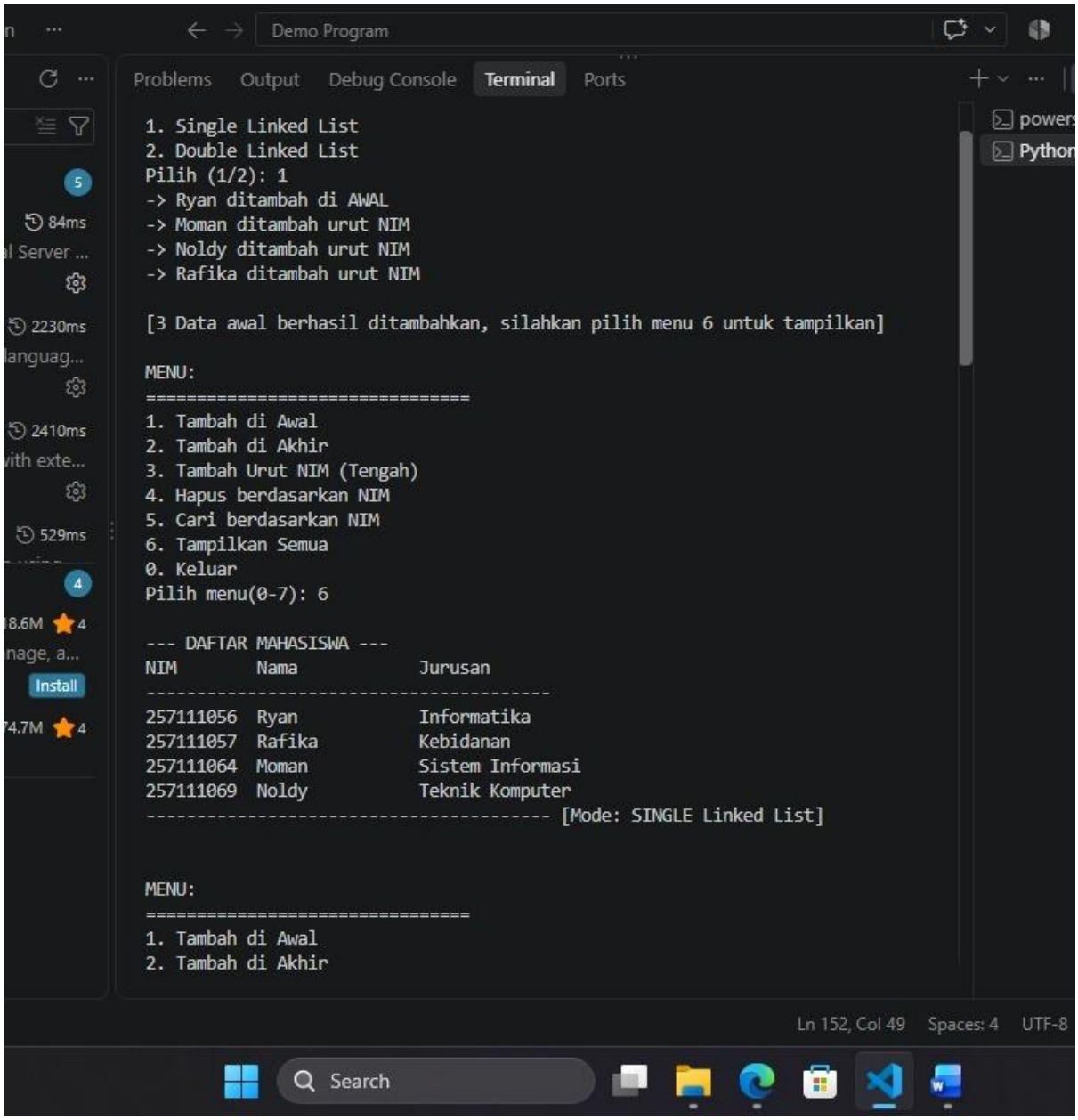
    while True:
        print("\nMENU:")
        print("================================")
        print("1. Tambah di Awal")
        print("2. Tambah di Akhir")
        print("3. Tambah Urut NIM (Tengah)")
        print("4. Hapus berdasarkan NIM")
        print("5. Cari berdasarkan NIM")
        print("6. Tampilkan Semua")
        print("0. Keluar")

        menu = input("Pilih menu(0-7): ")

        if menu == "0":
            break
        elif menu in ["1","2","3"]:
            nim = int(input("NIM: "))
            nama = input("Nama: ")
            jurusan = input("Jurusan: ")
            if menu == "1": ll.tambah_awal(nim, nama, jurusan)
            elif menu == "2": ll.tambah_akhir(nim, nama, jurusan)
```

```
            else: ll.tambah_urut_nim(nim, nama, jurusan)
        elif menu == "4":
            nim = int(input("NIM yang mau dihapus: "))
            ll.hapus(nim)
        elif menu == "5":
            nim = int(input("NIM yang dicari: "))
            ll.cari(nim)
        elif menu == "6":
            ll.tampilkan()
```

HASILNYA



Analisis Kompleksitas Double Linked List

1. Tambah di Awal

- Operasi: Membuat node baru, menghubungkan ke head, update pointer prev.
- Waktu: (O(1)) → konstan, karena tidak perlu traversal.
- Space: (O(1)) → hanya alokasi node baru.

2. Tambah di Akhir

- Operasi: Membuat node baru, menghubungkan ke tail, update pointer prev.
- Waktu: (O(1)) → langsung akses tail.
- Space: (O(1)).

3. Tambah Urut NIM

- Operasi: Traversal dari head hingga posisi yang sesuai, lalu sisipkan node.
- Waktu: (O(n)) → karena bisa butuh traversal sepanjang list.
- Space: (O(1)).

4. Hapus berdasarkan NIM

- Operasi: Cari node dengan NIM tertentu, update pointer prev dan next.
- Waktu: (O(n)) → pencarian linear.
- Space: (O(1)).

5. Cari berdasarkan NIM

- Operasi: Linear search dari head ke tail.
- Waktu: (O(n)).
- Space: (O(1)).

6. Tampilkan Semua

- Operasi: Traversal seluruh node untuk print data.
- Waktu: (O(n)).
- Space: (O(1)).

Ringkasan Kompleksitas

Operasi	Waktu	Space
Tambah di Awal	(O(1))	(O(1))
Tambah di Akhir	(O(1))	(O(1))
Tambah Urut NIM	(O(n))	(O(1))
Hapus berdasarkan NIM	(O(n))	(O(1))
Cari berdasarkan NIM	(O(n))	(O(1))
Tampilkan Semua	(O(n))	(O(1))

Insight

- Keuntungan Double Linked List dibanding Single Linked List adalah operasi hapus lebih efisien (tidak perlu traversal untuk mencari prev karena sudah ada pointer prev).
- Namun, pencarian tetap linear ((O(n))), sehingga untuk dataset besar lebih baik gunakan struktur data lain seperti hash table atau balanced tree.